

Pandas - cheatsheet

1. Importing pandas

```
import pandas as pd
```

2. Reading Data from csv file

```
df = pd.readcsv(filename)
```

3. Creating Test Data Objects

a. Creating DataFrame from random data of 5 columns and 20 rows

```
df = pd.DataFrame(np.random.rand(20, 5))
```

b. Creating a Series from list

```
df = pd.Series(my_list)
```

4. Read csv file into DataFrame

```
df = pd.readcsv("bmi.csv")
```

5. Exploring DataFrame

a. access 1st 5 rows

```
df.head()
```

b. access Last 5 rows

```
df.tail()
```

c. access 10 random rows

```
df.sample(n=10)
```

d. Structure of DataFrame

```
df.info()
```

e. Shape of DataFrame

```
df.shape
```

f. Get DataFrame's data into numpy array

```
df.values
```

g. Get DataFrame's column names

```
df.columns
```

f. Get index of DataFrame

```
df.index
```

Statistical summary

a. Statistical Summary of DataFrame

```
df.describe()
```

Include only non-numeric columns also

```
df.describe(include="all")
```

6. Accessing single or multiple columns

a. access single column

```
df["Height"]
```

b. access multiple columns

```
df[["Height", "Weight"]]
```

7. Filtering

Filtering based on numeric values

a. Return Boolean Mask of Height column greater than 165

```
df["Height"] > 165
```

b. Accessing rows with Height column greater than 165

```
df[df["Height"] > 165]
```

Filtering based on non-numeric values i.e. Categories

a. Accessing rows with Gender columns is equal to "Male"

```
df[df["Gender"] == "Male"]
```

Filtering based on multiple conditions (by multiple columns)

a. Accessing rows with Gender columns is equal to "Male" and having Height equals to 165

```
# Returns Boolean Mask
is_male = df["Gender"] == "Male"
is_height_eq165 = df["Height"] == 165
```

```
# now filter
df[is_male & is_height_eq165]
```

Filtering based on multiple numeric values in column

```
# Returns Boolean Mask
# Check if Height column contains 165 and 185 heights
is_height = df["Height"].isin([165, 185])
```

```
# now filter
df[is_height]
```

8. Sorting

Sorting by single column in ascending order

```
df.sort_values("Height")
```

Sorting by single column in descending order

```
df.sort_values("Height", ascending=False)
```

Sorting by multiple columns in ascending order

```
df.sort_values(["Height", "Weight"])
```

Sorting by multiple columns in descending order, Height column in ascending and Weight column in descending

```
df.sort_values(["Height", "Weight"], ascending=[True, False])
```

9. Creating New Column in DataFrame (Feature Engineering)

Creating a new column named "Height_m", with Height in meters

```
df["Height_m"] = df["Height"]/100
```

10. Indexing rows and columns

Label based Indexing - .loc

Selecting a single row

```
df.loc["row_label"]
```

Selecting a multiple rows

```
df.loc[["row_labelA", "row_labelB"]]
```

Selecting all rows with specific columns

```
df.loc[:, ["Height", "Weight"]]
```

Integer location based Indexing - .iloc

Selecting 2nd row

```
df.iloc[2]
```

Selecting a multiple rows - by integer indexing

```
# access 0th, 3rd and 9th rows
df.iloc[[0,3,9]]
```

Selecting rows and columns - using slicing

```
# access 3rd to 5th rows and 1st to 3rd column
df.iloc[3:5, 1:3]
```

11. Statistics (Aggregating methods)

Return mean of each columns

```
df.mean()
```

Return median of each columns

```
df.median()
```

Return number of non-nun values in each columns

```
df.count()
```

Return highest value in each columns

```
df.max()
```

Return lowest value in each columns

```
df.min()
```

Return standard deviation in each column

```
df.std()
```

Return variance in each column

```
df.var()
```

Column specific aggregation

```
df["Height"].var()
```

12. Grouping

Create groups of DataFrame by Genders

```
df.groupby("Gender")
```

13. Merging

inner join (Default)

```
pd.merge(left_df, right_df, on='id')
```

left join

```
pd.merge(left_df, right_df, on='id', how='left')
```

right join

```
pd.merge(left_df, right_df, on='id', how='right')
```

outer join

```
pd.merge(left_df, right_df, on='id', how='outer')
```

merging columns with different names

```
pd.merge(left_df, right_df, left_on='left_id', right_on='right_id', how='outer')
```

merging using multiple columns or keys

```
pd.merge(left_df, right_df, on=['year', 'country'], how='left')
```

14. Joining (Index-based join)

join on index (left) → default

```
df1.join(df2)
```

specify join type

```
df1.join(df2, how='inner')
```

```
df1.join(df2, how='outer')
```

```
df1.join(df2, how='right')
```

join on a column

```
df1.join(df2.set_index('id'), on='id')
```

Join multiple DataFrames at once

```
df1.join([df2, df3])
```

15. concatenation

Vertical stacking (Default)

```
pd.concat([df1, df2])
```

Horizontal stacking

```
pd.concat([df1, df2])
```

16. Data Cleaning

Inspect the Data

```
df.info()
```

Check duplicate rows

```
df.duplicated().sum()
```

Remove duplicate rows

```
df.drop_duplicates(inplace=True)
```

Check missing values per column

```
df.isnull().sum()
```

Drop/remove rows with missing values

```
df.dropna(inplace=True)
```

Drop/remove rows with missing values in specified columns

```
df.dropna(subset=['Name', 'Gender'], inplace=True)
```

Replace missing value in a column

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

Rename columns

```
df.rename(columns={'Name': 'Employee_Name', 'Comment': 'Performance'}, inplace=True)
```

Changing DataTypes

```
df['Age'] = df['Age'].astype(int)
```

Convert column to numeric DataTypes

```
df['col_str'] = pd.to_numeric(df['col_str'], errors='coerce')
```

